ספר פרויקט

משחק דמקה ב־Java ובּ־JavaFX

תיעוד תכנון, מימוש והפעלה

משחק רשת TCP, חשבונות ופרופיל משתמש

מגישים:

רביע אבו עאד — ת"ז 213984214

מותנא פרחאת — ת"ז 325079143

מנחה: ענבל רונן

אוקטובר 2026

תקציר ותוכן עניינים

הפרויקט הוא משחק דמקה ב־Java וב־JavaFX, הכולל משחק מקומי מול BOT ומשחק בין שני לקוחות דרך שרת TCP. השרת מנהל חדרים, מאמת מהלכים ושומר חשבונות ותוצאות ב־SQLite. הפרופיל מציג סטטיסטיקה והיסטוריית משחקים לכל משתמש מחובר.

הספר מרכז את חוקי המשחק, מבנה המערכת, התקשורת, אלגוריתם ה־BOT, החשבונות והממשק. ההסברים והתרשימים של הגרסה הארוכה אוחדו כדי להציג כל נושא במקום אחד.

1. מבוא, מטרות ודרישות 3

2. כללי המשחק ומימוש מהלך 4

3. מבנה המערכת ואחריות המחלקות 5

4. תקשורת TCP וניהול חדרים 7

5. השחקן הממוחשב — BOT 11

6. חשבונות, SQLite ופרופיל 13

7. ממשק המשתמש ומסכי המערכת 15

8. בדיקות ואימות המערכת 17

9. הפעלה ומדריך משתמש 18

10. שימוש בכלי AI והתרומה האישית 19

11. סיכום, אתגרים ומקורות 20

1. מבוא, מטרות ודרישות

1.1 רקע והרחבת הפרויקט

דמקה היא משחק שבו ההחלטה על מהלך תלויה במיקום הכלים, באפשרויות האכילה ובתגובת היריב. הגרסה הראשונה הייתה מקומית: שני אנשים השתמשו באותו מחשב. הגרסה הנוכחית מאפשרת לכל שחקן להפעיל לקוח JavaFX משלו ולהתחבר לשרת משותף. המשחק מול BOT נשאר מקומי וזמין גם ללא שרת.

1.2 מטרות

- להציג לוח ברור, בחירת כלי ומשבצת יעד, אנימציה ומשוב על מהלך שגוי.
- לרכז את חוקי הדמקה ב־Game ולהשתמש בהם גם בשרת וגם במשחק מול BOT.
- לאפשר יצירת חדר, שיתוף קוד, הצטרפות וסנכרון בין שני שחקנים.
- להוסיף הרשמה, התחברות ופרופיל עם סטטיסטיקה והיסטוריה שנשמרות לאורך זמן.

1.3 היקף

משחק הרשת מיועד למחשבים באותה רשת מקומית. כתובת השרת והפורט ניתנים להגדרה בזמן ההפעלה. אין כרגע פריסה בענן, שעון משחק או תיקו אוטומטי. חשבונות ותוצאות נשמרים בשרת; מצב חדר פעיל נמצא בזיכרון בלבד. משתמש ללא חשבון פעיל יכול לתרגל מול BOT, אך התוצאה אינה נרשמת.

1.4 תוצר וסביבת פיתוח

התוצר כולל לקוח גרפי, שרת Java, קובץ SQLite ובדיקות אוטומטיות. הפרויקט משתמש ב־Java 17, JavaFX 17.0.6 ו־Gradle 8.8. Main מפעילה את הלקוח; DamkaServer מפעילה את השרת. קובצי המקור נמצאים ב־src/main/java, העיצוב ב־damka.css והבדיקות ב־src/test/java.

2. כללי המשחק ומימוש מהלך

2.1 מצב פתיחה ותנועה

הלוח בגודל 8×8, עם 12 כלים לכל שחקן על המשבצות הכהות. X מתחיל בתחתית, נע מעלה ופותח את המשחק; O מתחיל למעלה ונע מטה. הקואורדינטות הן בטווח 0 עד 7. כלי רגיל נע משבצת אחת באלכסון קדימה, ואוכל בקפיצה של שתי משבצות מעל יריב אל יעד פנוי. כלי רגיל אוכל קדימה בלבד.

2.2 אכילה מחייבת והכתרה

אם קיימת אכילה, צעד רגיל אינו מותר. כאשר אפשר לאכול שוב אחרי קפיצה, ממשיכים עם אותו כלי ואותו שחקן נשאר בתור. כלי x בשורה 0 וכלי o בשורה 7 מוכתרים למלכים. מלך נע ואוכל לשני הכיוונים, אך אינו מעופף: צעד של משבצת אחת או קפיצת אכילה של שתי משבצות. ההכתרה נעשית לפני בדיקת ההמשך, ולכן מלך חדש עשוי להמשיך לאכול לאחור.

2.3 Game היא הסמכות לכללים

getValidMoves מחזירה את האפשרויות החוקיות: אכילות בלבד כאשר קיימת אכילה, או צעדים רגילים אחרת. באמצע רצף היא מגבילה את האפשרויות למיקום captureContinuationPosition. Game.move מחפשת את המקור והיעד ברשימה; מהלך שנדחה אינו משנה את הלוח או התור.

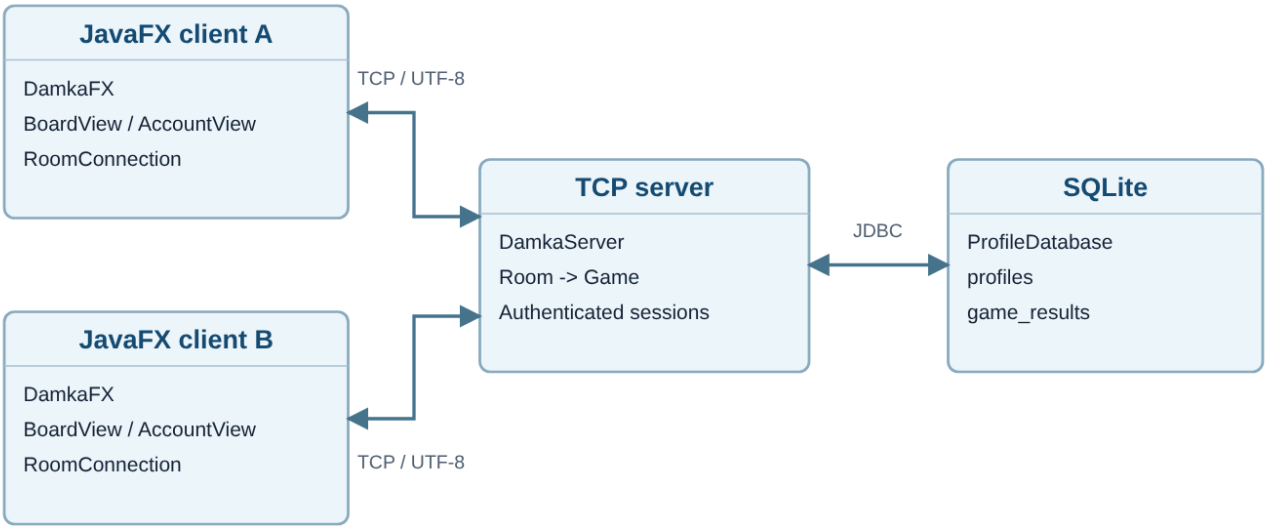
אחרי אישור Game מסירה כלי שנאכל, מזיזה את הכלי, מכתירה, שומרת lastMove ומשאירה חובת המשך או מחליפה תור. Game.Move כוללת מקור, יעד, סימן כלי ומיקום אכילה לצורך תצוגה. Board מבצעת גישה והזזה של הנתונים, בלי להחליט על חוקיות.

2.4 ניצחון

getWinnerSymbol מחזירה X או O כאשר ליריב אין מהלך חוקי, ו־'\0' כאשר אין מנצח. היעדר כלים גורם גם להיעדר מהלכים. פרישה, עזיבה וניתוק במשחק רשת מטופלים ב־Room, ולא כחוק תנועה של Game.

3. מבנה המערכת ואחריות המחלקות

3.1 שרת, לקוחות ושמירה



איור 1 — כל שחקן מפעיל לקוח; מצב המשחק המקוון והמסד נמצאים בשרת.

DamkaFX מנהלת קלט, מצב מסך ותגובות להודעות. BoardView מציירת ומנפישה, AccountView מציגה הזדהות ו־ProfileWindow מציגה פרופיל. RoomConnection מבצעת תקשורת ברקע. DamkaServer מקבלת לקוחות ומנתבת בקשות. Room מחזיקה משחק מקוון ושני משתתפים, ו־ProfileDatabase קוראת וכותבת SQLite.

3.2 שני מצבי משחק

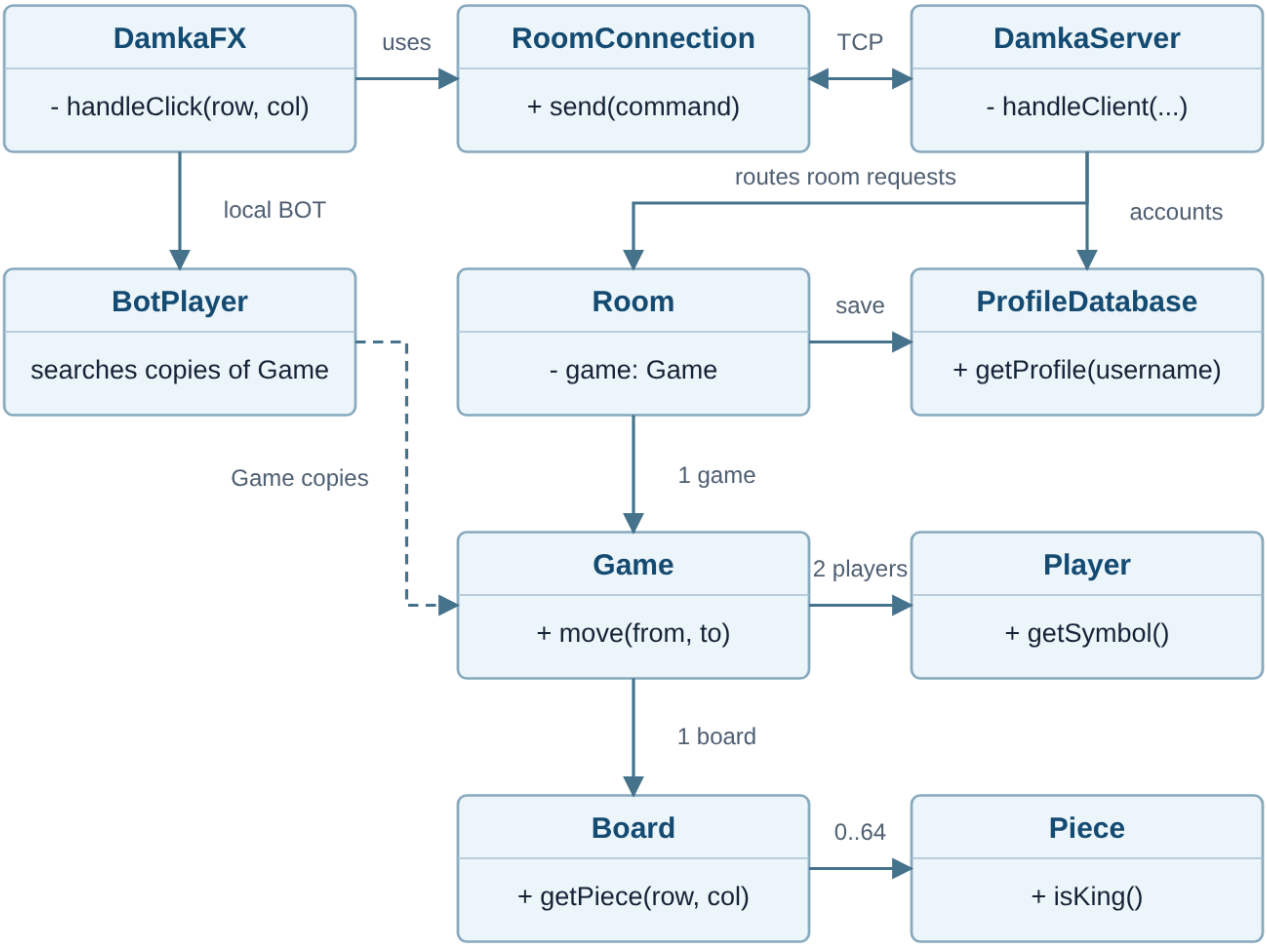
מול אדם אחר Game נמצאת בתוך Room בשרת, והלקוח מחזיק תמונת לוח לציור בלבד. מול BOT קיימת botGame מקומית ב־DamkaFX. BotPlayer מחפשת מהלך על עותקי Game. בשני המצבים משתמשים באותם כללים ובאותו BoardView.

3.3 הפרדה בין סוגי בדיקות

Game בודקת תנועה, אכילה, הכתרה וניצחון. Room בודקת מי שלח בקשה, חדר פעיל, תור ומבנה נתונים. הממשק בודק בחירה וזמינות פעולה. ההפרדה מונעת שכפול של כללי המשחק ומאפשרת בדיקה ללא חלון גרפי.

3. מבנה המערכת ואחריות המחלקות — המשך

3.4 תרשים מחלקות



איור 2 — הקשרים המרכזיים בין הממשק, התקשורת, המשחק והשמירה.

Position שומרת שורה ועמודה; Piece שומרת סימן, ואות גדולה מציינת מלך. Player היא שחקן במודל המשחק ואינה חשבון במסד. ProfileData היא רשומת נתוני פרופיל להעברה לתצוגה. החץ TCP מייצג קשר בין תהליכים, ולא קריאה ישירה למתודה על אובייקט משותף. התרשים מציג את הקשרים המרכזיים, ולא את כל מחלקות הספריות.

4. תקשורת TCP וניהול חדרים

4.1 ServerSocket ו־Socket

TCP מעבירה רצף בתים מסודר בין שני קצוות. ServerSocket מאזינה לחיבורים בפורט השרת. accept ממתינה ללקוח ומחזירה Socket נפרדת של החיבור. השרת מפעיל Thread שקורא handleClient עבור אותו לקוח, וחוזר לקבל חיבורים נוספים. שני השחקנים מתחברים לשרת, ולא ישירות זה לזה.

4.2 פתיחת החיבור ושימוש חוזר

בעת הרשמה או כניסה DamkaFX.submitAuthentication יוצרת RoomConnection. connect מפעילה קורא ברקע, ו־readMessages קוראת socket.connect עם כתובת ופורט. אחרי AUTH_OK אותו חיבור משמש חדרים, מהלכים, פרופיל ותוצאות .BOT אין Socket חדשה לכל לחיצה.

4.3 קריאה וכתיבה

רכיב	תפקיד
InputStreamReader	ממיר בתים לתווים בקידוד .UTF-8
BufferedReader	קורא הודעות באמצעות .readLine
OutputStreamWriter	ממיר תווים לבתים.
PrintWriter	שולח שורת הודעה באמצעות .println

כל הודעה מסתיימת בשורה חדשה. readLine ממתינה לשורה או לסיום הזרם ולכן פועלת ברקע. send משתמשת בעובד כתיבה יחיד לשמירת סדר הפקודות. Platform.runLater מעבירה את עדכון המסך לחוט .JavaFX

> localhost הוא המחשב שבו הלקוח רץ. למשחק בין מחשבים משתמשים בכתובת LAN של השרת ובאותו פורט. חמש השניות בקריאת connect מגבילות הקמת חיבור בלבד; אין הגבלת זמן לחשיבה בין מהלכים.

4. תקשורת TCP וניהול חדרים — המשך

4.4 יצירה והצטרפות

Room מאגדת Game אחת, שני חיבורים, שמות משתמש וקוד הזמנה בן שישה תווים. החדרים נמצאים במפת HashMap בזיכרון. היוצר מקבל PLAYER X. משתמש אחר מצטרף בעזרת הקוד ומקבל PLAYER O. ROOM_JOINED מאשרת את ההצטרפות; ROOM_READY מודיעה לשניהם שהמשחק יכול להתחיל.

בקשה או אירוע	תגובה מרכזית
CREATE_ROOM	ROOM_CREATED <code>, PLAYER X
JOIN_ROOM <code>	ROOM_JOINED <code>, PLAYER O
השחקן השני התקבל	ROOM_READY, BOARD, TURN X לשניהם
MOVE <fromRow> <fromCol> <toRow> <toCol>	MOVE_APPLIED, BOARD, TURN או ERROR
RESIGN / LEAVE_ROOM	GAME_OVER / ROOM_LEFT לפי מצב החדר

4.5 העברת מצב הלוח

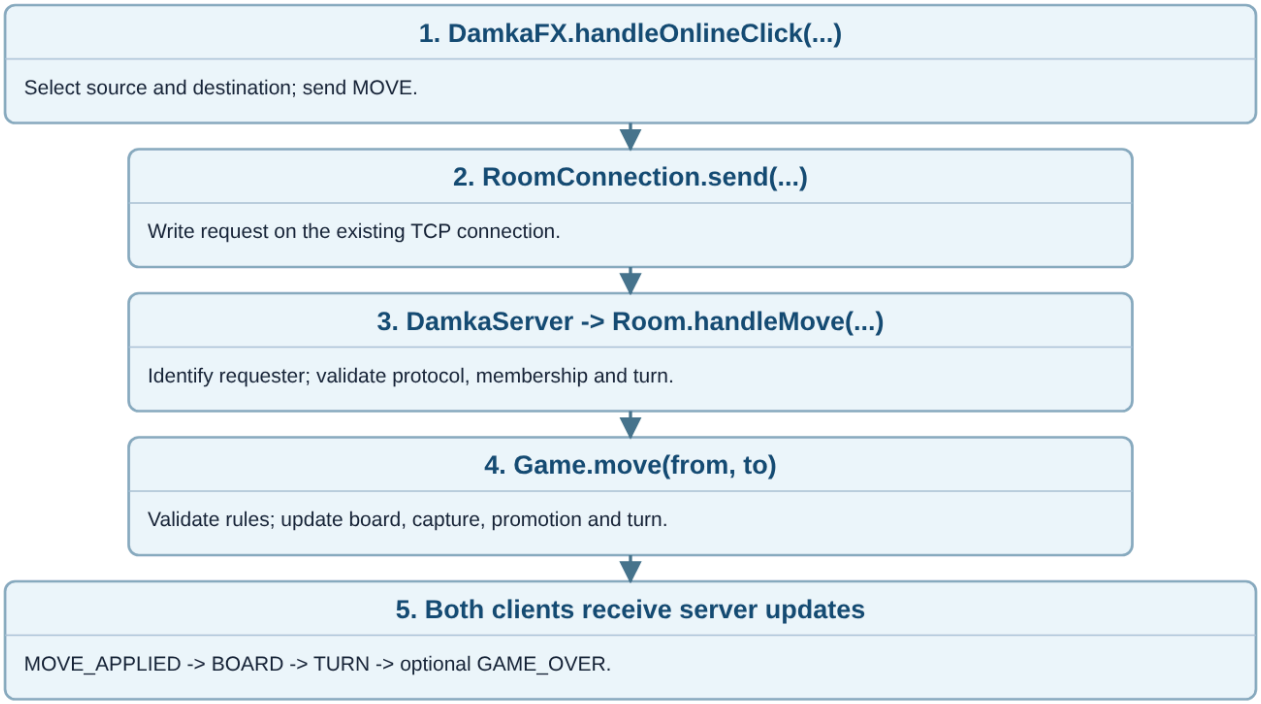
Room.encodeBoard עוברת על 64 התאים לפי שורות. נקודה מייצגת תא ריק, x/o כלי רגיל ו־X/O מלך. השרת שולח BOARD עם המחרוזת. הלקוח מוצא תא באמצעות row × 8 + col. זהו snapshot לציור: הלקוח אינו משנה את המחרוזת כדי לקבוע מהלך חדש.

```
MOVE 5 0 4 1
```

הדוגמה מבקשת להזיז כלי מ־(5,0) אל (4,1). זהות המבקש נגזרת מה־Socket ומהחשבון המאומת, ואינה שדה שהלקוח יכול לבחור בתוך MOVE. שני המושבים בחדר חייבים להשתייך לחשבונות שונים.

4. תקשורת TCP וניהול חדרים — המשך

4.6 מהלך אחד מתחילתו ועד התצוגה



איור 3 — מסלול הבקשה והעדכון לשני הלקוחות.

handleOnlineClick בוחרת מקור ויעד. sendOnlineMove בונה בקשה, ו־RoomConnection.send שולחת אותה. DamkaServer.handleClient מעבירה ל־Room.handleMove. החדר בודק משתתף, תור וקואורדינטות בטווח 0 עד 7; Game.move קובעת את חוקיות המשחק.

4.7 אישור, דחייה ואנימציה

מהלך שגוי מחזיר ERROR למבקש בלי לשנות את הלוח. אחרי אישור שני הלקוחות מקבלים MOVE_APPLIED, אחריה BOARD ואחריה TURN. בהיעדר אכילה קואורדינטות הכלי שנאכל הן -1,-1. Game מספקת את פרטי התנועה; השרת מספק את המצב המחייב.

incomingMessages בלקוח ממתינה לסיום האנימציה לפני עיבוד BOARD ו־TURN הבאות. בזמן קבלת לוח מתאפס התור זמנית עד הודעת TURN, כדי למנוע שימוש בלוח חדש עם תור ישן.

4. תקשורת TCP וניהול חדרים — המשך

4.8 סנכרון בשרת

לכל לקוח עובד נפרד, ולכן בקשות עשויות להגיע במקביל. synchronized (Room.class) מגן על מפת החדרים המשותפת. נעילת אובייקט Room מגינה על המשתתפים ועל Game של אותו חדר. כך שני מצטרפים אינם תופסים אותו מקום, ושני מהלכים אינם משנים יחד את אותו תור. הסרה מהמפה נעשית אחרי שחרור נעילת החדר.

4.9 סיום משחק

אירוע	תוצאה
ניצחון לפי Game	GAME_OVER עם סיבה .RULES
RESIGN	ניצחון ליריב, סיבה .RESIGNATION
LEAVE_ROOM במשחק פעיל	ניצחון ליריב, סיבה .OPPONENT_LEFT
ניתוק שהתגלה	ניצחון ליריב, סיבה .OPPONENT_DISCONNECTED
עזיבת חדר שממתין לשחקן שני	החדר נסגר בלי תוצאת משחק.

תוצאה מקוונת נרשמת פעם אחת ב־SQLite. אם השמירה נכשלת, נשלחת RESULT_ERROR. סיום חדר מסיר אותו מהמפה, אך יכול להשאיר את חיבור החשבון פעיל למשחק נוסף.

4.10 סגירה ומגבלות

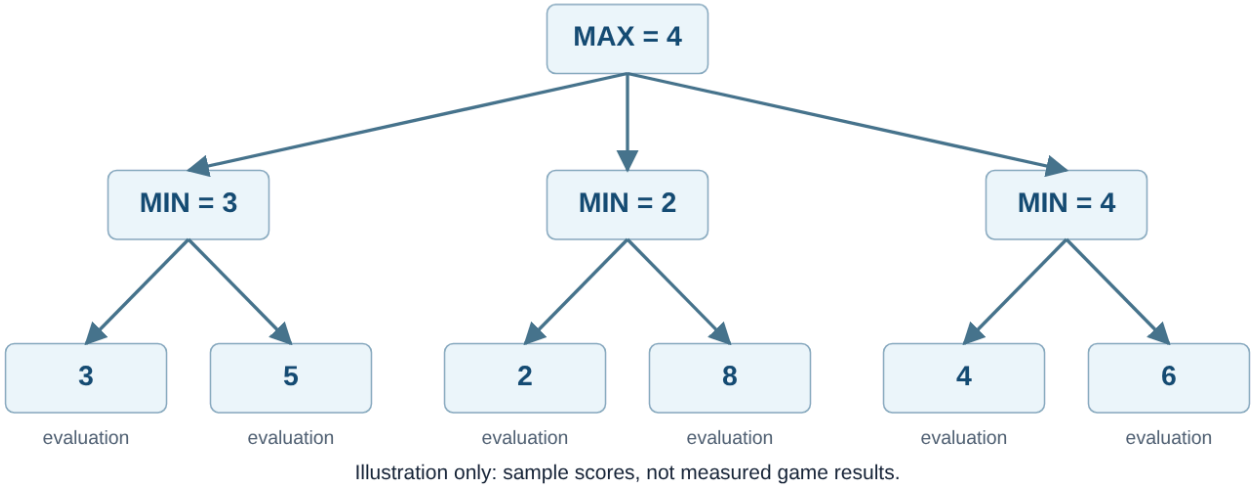
try (Socket connection = client) מתייחסת לאותו חיבור ומבטיחה close ביציאה מהבלוק, גם בחריגה. סגירה בלקוח משחררת readLine שממתינה. חוטי RoomConnection הם daemon כדי שלא יחזיקו את היישום פתוח לבד. אין heartbeat או חידוש חדר לאחר ניתוק; ניתוק שקט עשוי להתגלות באיחור. TCP אינה מצפינה את ההודעות.

5. השחקן הממוחשב — BOT

5.1 חיפוש מהלך

BotPlayer.findBestMove מקבלת מצב Game, סימן BOT ועומק. לכל מהלך חוקי היא יוצרת new Game(game), מבצעת בו את המהלך ומפעילה Minimax. המהלך בעל הציון הגבוה ביותר נבחר; בשוויון נשמרת האפשרות הראשונה. ההעתקה כוללת לוח וכלים נפרדים, תור ומיקום המשך אכילה, ולכן החיפוש אינו משנה את המשחק שמוצג.

5.2 Minimax



איור 4 — דוגמת חיפוש: MAX בוחרת 4 מתוך תוצאות MIN. הציונים הם המחשה.

בצומת שבו תור BOT נבחר הערך המרבי, ובצומת של היריב נבחר המזערי. זהות השחקן נקראת מתוך Game בכל מצב; רצף אכילות יכול להשאיר אותו שחקן כמה שכבות. ניצחון של ה־BOT מקבל 10000 והפסד שלו 10000-. כאשר העומק נגמר בלי סיום, מופעלת הערכת הלוח.

5.3 גיזום Alpha-Beta

alpha ו־beta הם גבולות התוצאות שכבר נמצאו. כאשר beta קטנה או שווה ל־alpha, יתר ענף החיפוש אינו יכול לשפר את הבחירה, ולכן אין צורך לבדוק אותו. בחיפוש מלא עם b אפשרויות ועומק d מספר המצבים הוא מסדר גודל O(b^d); גיזום יכול לצמצם עבודה, בהתאם למצבים ולסדר האפשרויות.

5. השחקן הממוחשב — BOT — המשך

5.4 קושי: אותו אלגוריתם, עומק שונה

רמה	עומק	השפעה
Easy	1	בודקת את תוצאת המהלך המועמד.
Medium	2	בודקת שכבה נוספת של מהלכים.
Hard	3	בודקת יותר אפשרויות, עם יותר עבודה.

שלוש הרמות משתמשות באותו Minimax ובאותו Alpha-Beta. עומק פוחת בכל מהלך או קפיצה, ולכן אינו בהכרח מספר תורות מלאים לסירוגין. אין מנגנון למידת מכונה.

5.5 ניקוד הלוח

```
score = 3 × (BOT men − opponent men)
      + 6 × (BOT kings − opponent kings)
```

כלי רגיל שווה 3 ומלך 6. מלך מקבל משקל גבוה כי הוא יכול לנוע ולאכול בשני הכיוונים. יחס 2:1 הוא בחירת תכנון פשוטה; לא תועד ניסוי המוכיח שהוא אופטימלי. לדוגמה, יתרון של כלי רגיל אחד כשמספר המלכים זהה נותן ציון 3.

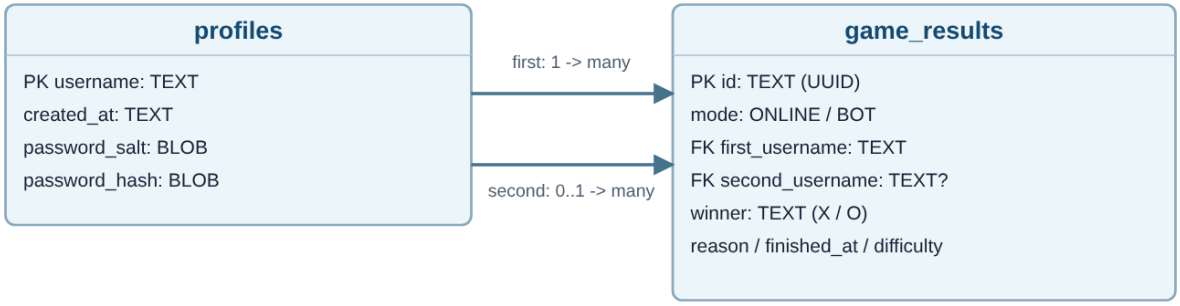
5.6 שילוב ומגבלות

האדם משחק X וה־BOT משחק O. לאחר מהלך האדם והאנימציה, הממשק מפעיל BOT אם הגיע תורו. גם המהלך שלו עובר דרך Game.move, והוא ממשיך עם אותו כלי כאשר יש רצף אכילות. ההערכה מתייחסת לחומר בלבד, בלי ניקוד למרכז, הגנה או ניידות. חיפוש BOT הנוכחי נקרא מחוט JavaFX, ולכן חיפוש עמוק יותר עלול לפגוע בתגובה; פעולות הרשת מתבצעות ברקע.

6. חשבונות, SQLite ופרופיל

6.1 נתונים מתמשכים בשרת

ProfileDatabase משתמשת ב־JDBC וב־SQLite. קובץ data/damka.db נשאר אחרי הפעלה מחדש של השרת. כל פעולה פותחת Connection ונסגרת בסיום. PreparedStatement מפרידה SQL מפרמטרי המשתמש.



איור 5 — profiles מחזיקה חשבונות, ו־game_results מחזיקה תוצאות.

6.2 סכמה

טבלה	שדות עיקריים
profiles	username (PK), created_at, password_salt, password_hash
game_results	id (PK), mode, first_username / second_username (FK), winner, reason, finished_at, difficulty

6.3 הרשמה והתחברות

השם כולל 1 עד 24 אותיות, ספרות, _ או -, והסיסמה 8 עד 128 תווים. הטופס בודק אישור סיסמה בהרשמה; השרת מאמת שוב. signUp יוצרת salt אקראי בן 16 בתים ו־hash מסוג PBKDF2WithHmacSHA256 עם 600,000 איטרציות ופלט 256 סיביות. הסיסמה המקורית אינה נשמרת. שם קיים אינו נדרס. login שולפת salt ו־hash, מחשבת עם הסיסמה שנמסרה ומשווה באמצעות MessageDigest.isEqual.

AUTH_OK קושרת את username לחיבור, וגם הרשמה מוצלחת נכנסת לחשבון. בקשות לחדרים ולפרופיל משתמשות בזהות הזו. Sign out סוגרת את החיבור; שינוי חשבון מחייב חיבור חדש. Base64 היא קידוד הפיך ולא הצפנה, והתקשורת הנוכחית אינה מוגנת ב־TLS.

6. חשבונות, SQLite ופרופיל — המשך

6.4 שמירה פעם אחת

עם הצטרפות השחקן השני נוצר matchId מסוג UUID. בסיום משחק מקוון נשמרת שורה אחת עם המשתתפים, המנצח, הסיבה וזמן הסיום. מזהה id ייחודי מונע ספירה כפולה: חזרה על אותה תוצאה אינה מוסיפה משחק, ושימוש באותו id לתוצאה אחרת נדחה. חדר המתנה שנסגר אינו נרשם.

6.5 סטטיסטיקה והיסטוריה

getProfile קוראת את התוצאות שבהן החשבון הוא המשתתף הראשון או השני. לפי המושב ו־winner נקבע Win או Loss. Online ו־BOT נספרים בנפרד. Played, Wins ו־Losses כוללים את כל התוצאות; ההיסטוריה ממוינת מהחדש לישן ומציגה עד 100 משחקים. שניהם מחושבים מאותה קריאה למסד.

בקשה	תשובה ומשמעות
SIGNUP / LOGIN	AUTH_OK או AUTH_ERROR עבור פרטי החשבון.
PROFILE_REQUEST	PROFILE עם ProfileData מקודדת, או PROFILE_ERROR.
BOT_RESULT <id> <WIN|LOSS> <difficulty> <time>	BOT_SAVED או BOT_ERROR. זהות החשבון נלקחת מהחיבור.

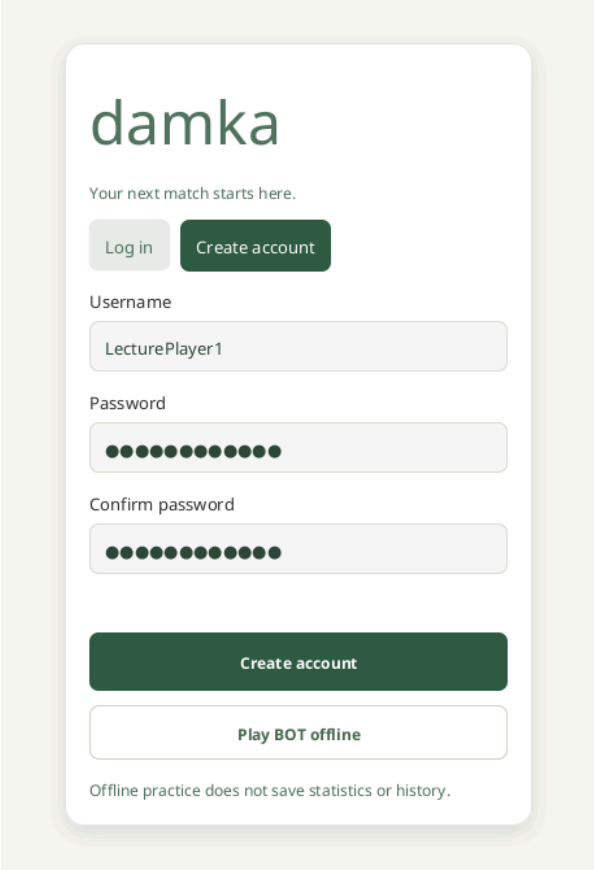
ProfileWindow מציגה את הנתונים. פתיחת החלון או Refresh שולחות PROFILE_REQUEST בלי שם משתמש לבחירה. השרת קורא את הפרופיל של החשבון המאומת, והלקוח מפענח באמצעות ProfileData.decode. סגירת חלון הפרופיל משאירה את חלון המשחק פתוח.

6.6 מתי תוצאת BOT נשמרת

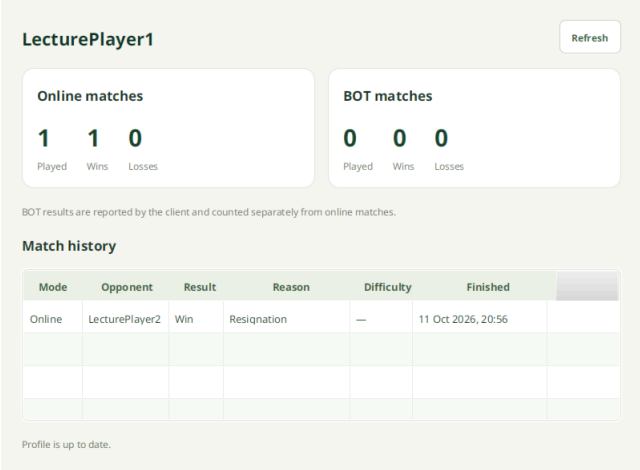
המשחק חייב להתחיל ולהסתיים באותו חשבון פעיל ובאותו אובייקט חיבור. התחברות מאוחרת או ניתוק והתחברות מחדש אינם שומרים משחק אימון קודם. BOT מנוהלת מקומית, והשרת אינו מריץ אותה מחדש לאימות התוצאה; לכן הסטטיסטיקה שלה נפרדת. בלי שרת או חשבון אין שמירת תוצאות, אין תור מקומי ואין העלאה מאוחרת.

7. ממשק המשתמש ומסכי המערכת

7.1 הזדהות ופרופיל



איור 6 — הרשמה בממשק הנוכחי.



איור 7 — פרופיל אחרי משחק הדגמה.

AccountView מציעה Log in, Create account ו־Play BOT offline. הצלחת הזדהות מובילה למסך המשחק. Profile פותח חלון עם שתי קבוצות סטטיסטיקה וטבלת היסטוריה; Refresh מבקש מידע חדש. הצילומים הופקו עם חשבונות הדגמה במסד זמני, ולא עם נתוני שחקנים אמיתיים.

7.2 בחירה, משוב ועיצוב

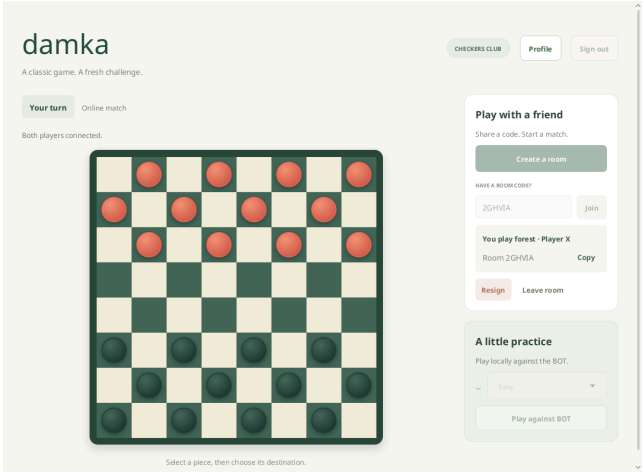
לחיצה על כלי בוחרת אותו, ולחיצה על יעד פנוי מבקשת מהלך. BoardView מציגה לוח קרם וירוק, כלי X בירוק וכלי O באלמוג. בחירה מסומנת בטבעת זהובה ומלך במסגרת עבה. שגיאה מופיעה בכרטיס צף שניתן לסגור; היא אינה מקטינה את הלוח, ופעולה מוצלחת מנקה הודעה קודמת. העיצוב נמצא ב־damka.css.

7.3 חלון גמיש

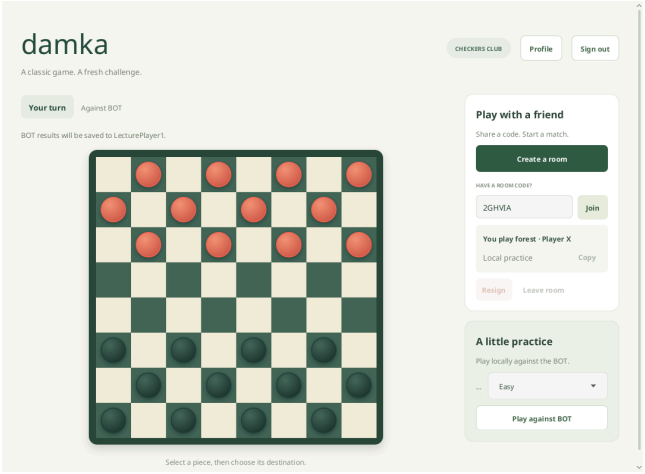
הלוח מתאים את גודלו לחלון. בחלון צר כרטיסי המשחק וה־BOT עוברים מתחתיו, וקיימת גלילה כשצריך. קוד החדר ניתן להעתקה באמצעות Copy. כתובת השרת והפורט מוגדרים בהפעלה ואינם שדות במסך.

7. ממשק המשתמש ומסכי המערכת — המשך

7.4 משחק רשת ומשחק BOT



איור 8 — חדר פעיל מול שחקן.



איור 9 — משחק BOT מקומי.

Create a room ו־Join משתמשות בחיבור המאומת. אחרי ששני המשתתפים מחוברים מוצגים הלוח והתור של השרת. Resign ו־Leave room מטפלות בסיום. Play against BOT מתחיל משחק מקומי; בוחרים Easy, Medium או Hard. הממשק מונע פעולה חדשה בזמן בקשה או אנימציה.

7.5 אותה אנימציה בשני המצבים

BoardView.animateMove מסתירה את הכלי במקור ומציגה כלי נע בשכבה מעל הלוח. Timeline מנפישה במשך כ־320 מילישניות; FadeTransition מעלימה כלי שנאכל. מרכזי התאים מחושבים בכל פריים כדי ששינוי גודל וגלילה לא יזיזו את מסלול התנועה. בסוף מוצגת התמונה המעודכנת. אותו רכיב משמש BOT ומשחק רשת.

7.6 היקף ההדגמה

הצילומים הופקו בממשק הנוכחי: הרשמה, יצירת חדר, הצטרפות לקוח פרוטוקול נוסף, שני מהלכים, מהלך שגוי, פרישת היריב ושליפת הפרופיל. זו הדגמה מקומית של התוכנה, ולא בדיקת שני מחשבים פיזיים ברשת.

8. בדיקות ואימות המערכת

8.1 בדיקות אוטומטיות

הבדיקות משתמשות ב־JUnit 5, בחיבורי TCP אמיתיים ב־loopback ובמסדי SQLite זמניים. הן אינן תלויות בחלון גרפי. הטבלה מרכזת את ההרצה שנבדקה בעת הכנת הספר:

מחלקה	בדיקות	מה נבדק
GameRulesTest	11	תנועה, אכילה מחייבת ורצף, הכתרה, העתקה וניצחון.
RoomProtocolTest	11	מהלכים, סנכרון, הצטרפות מתחרה, סיום ושמירת תוצאה.
RoomConnectionTest	4	סדר פקודות, תגובות, כשל חיבור וסגירה.
AuthProfileTest	14	הרשמה, כניסה, זהות, חדרים, פרופיל ושמירת BOT.
ProfileDatabaseTest	7	התמדה, סיסמאות, תוצאות ללא כפילות והיסטוריה.

8.2 תוצאות

```
bash gradlew --no-daemon --max-workers=2 test classes startScripts
```

בהרצה המתועדת מ־11.10.2026 עברו 47 בדיקות, עם 0 כישלונות, שגיאות או דילוגים. התקבל BUILD SUCCESSFUL, וגם classes ו־startScripts הסתיימו בהצלחה. הסביבה הייתה Debian 13.6, OpenJDK 21.0.12.1 ו־Gradle 8.8. יעד התאימות של הפרויקט הוא Java 17.

8.3 מגבלות האימות

בדיקות אוטומטיות מאמתות את הכללים, הפרוטוקול והנתונים; אינן מחליפות בדיקה חזותית של כל אנימציה ושינוי גודל. צילומי ההדגמה הופקו מול שרת ומסד זמניים. הכנת הספר אינה מוכיחה בדיקת Windows מלאה, שני מחשבים ברשת או פריסה בענן.

9. הפעלה ומדריך משתמש

9.1 במחשב אחד

פותחים את הפרויקט ב־VS Code וממתינים לייבוא Gradle. אפשר להפעיל Run מעל DamkaServer.main פעם אחת, ואז Run מעל Main.main פעמיים. לחלופין מפעילים מתוך תיקיית הפרויקט שלושה מסופי PowerShell:

```
# Terminal 1: server
.\gradlew.bat tcpServer

# Terminals 2 and 3: one client in each
.\gradlew.bat run
```

נכנסים לשני חשבונות שונים. הראשון יוצר חדר, מעתיק את הקוד והשני מצטרף. X מתחיל. בוחרים כלי ואז יעד. Profile מציגה תוצאות; סוגרים את החלון כדי לחזור למשחק. לתרגול ללא שרת מפעילים רק Main ובוחרים Play BOT offline.

9.2 רשת מקומית

מפעילים את השרת במחשב אחד, מוצאים את IPv4 שלו באמצעות ipconfig ומאפשרים את פורט TCP בחומת האש ברשת הפרטית. localhost מציין את המחשב שעליו הלקוח פועל. בכל לקוח מחליפים את הכתובת בדוגמה בכתובת השרת בפועל:

```
.\gradlew.bat run -PserverHost=192.168.1.10 -PserverPort=5000
```

לפורט אחר מפעילים tcpServer עם "6000"args=-- ואת הלקוחות עם PserverPort=6000.- ב־VS Code אפשר להשתמש ב־vmArgs של Ddamka.server.host- ו־Ddamka.server.port.-. ב־Linux/macOS משתמשים ב־bash gradlew.

9.3 בדיקות ותקלות

gradlew.bat test\. מפעילה בדיקות. אם Java אינה מזוהה במסוף, מגדירים JAVA_HOME לתיקיית JDK ואת bin ב־Path. הגדרת Java ב־VS Code אינה בהכרח הגדרת המסוף. בכשל חיבור בודקים שרת פעיל, כתובת, פורט וחומת אש; בשגיאת בנייה מתקנים לפני שמריצים גרסה ישנה.

10. שימוש בכלי AI והתרומה האישית

10.1 היקף ואופן השימוש

נעשה שימוש נרחב ב־Codex וב־GitHub Copilot ללמידה, תכנון, יצירת קוד, תיקונים, עיצוב, בדיקות ותיעוד. הסיוע כלל את המעבר ל־TCP, חיבור JavaFX לשרת, חדרים, חשבונות, SQLite והפרופיל. אין מדידה אמינה של אחוז הקוד שנוצר באמצעות AI, ולכן אינו מוצג אחוז משוער.

10.2 תרומה אישית בתהליך

מותנא הגדיר דרישות וקיבל החלטות על התנהגות המערכת, הוביל את הפיתוח בעזרת הכלים ובדק את היישום ב־Windows. הוא ביקש הסברים על Socket, Threads וזרימת ההודעות, קרא שינויים וביקש לפשט קוד או להתקדם בצעדים קטנים. העבודה עם AI כללה בחינת ההצעות והתאמתן לדרישות, ולא רק קבלת קוד ללא בדיקה.

10.3 דוגמה לבקרה על תוצאה

בבדיקה ידנית זוהתה הודעת מהלך לא חוקי שנשארה גם לאחר מהלך תקין. המשוב הוביל לשינוי טיפול ההודעות. נבדקו גם חיבורי לקוחות, יצירה והצטרפות, תורות ושינוי גודל. בדיקות JUnit מוסיפות אימות לכללים ולנתונים, אך אינן מחליפות בדיקה חזותית והבנת הקוד.

10.4 חלוקת העבודה בין המגישים

> להשלמה לפני ההגשה: פירוט חלוקת העבודה בין המגישים לפי המשימות שבוצעו בפועל. לצד תהליך הפיתוח המתואר לעיל אפשר לתעד בדיקות LAN, סקירת תיעוד והכנת מצגת, לאחר ביצוען.

11. סיכום, אתגרים ומקורות

11.1 סיכום

המערכת הורחבה ממשחק מקומי למערכת שרת ולקוחות עם חשבונות מתמשכים. השרת מנהל את המשחק המקוון, Game מרכזת את הכללים, והלקוחות מציגים את המצב ומבקשים פעולות. BOT פועלת מקומית באותו מודל, והפרופיל מבחין בין תוצאות מקוונות המאומתות בשרת לבין תוצאות BOT שמדווחות מהלקוח.

11.2 אתגרים ופתרונות

- רצף אכילות: שמירת המיקום המחויב בתוך Game ובעותקי החיפוש.
- פעולות רשת וממשק: קורא ברקע, עובד כתיבה יחיד ועדכון באמצעות Platform.runLater.
- אנימציה וסנכרון: תור הודעות שממתין לסיום התנועה לפני עדכון הלוח.
- תוצאות כפולות: UUID ושורה אחת לתוצאה, עם סטטיסטיקה שנגזרת מהשורות.

11.3 המשך פיתוח

אפשר להוסיף TLS והגבלת ניסיונות כניסה, שחזור משחק אחרי ניתוק, גילוי ניתוק שקט, חיפוש BOT בחוט רקע והערכה עשירה יותר. הרחבות אלה אינן חלק מהתוצר הנוכחי. יש מקום להרחיב גם בדיקות GUI ו־LAN.

11.4 מקורות

- קוד MuthannaFrhat/damka-game בענף add-user-profile, קובצי README ו־LEARNING ובדיקות JUnit.
- Damka_Project_Book_Corrected_v4.pdf והערות המנחה — בסיס למבנה הספר.
- openjfx.io. — docs.oracle.com; OpenJFX — Java SE API
- junit.org. — docs.gradle.org; JUnit — sqlite.org; Gradle — SQLite

הספר מתאר את המקורות שנבדקו ואת היקף האימות בפועל. תיאור התוכנה, הוראות ההפעלה והבדיקות מאפשרים לעקוב מהמסך אל השרת, אל Game ואל השמירה במסד.